

tradebot.py — Complete Documentation

Written for a systems programmer who thinks in C, not Python, and who has never traded. Trading concepts are explained from zero; Python idioms are mapped to what you already know from C/POSIX. Read top to bottom once, then use it as a reference while hacking.

Table of Contents

1. [What this bot actually does \(30-second version\)](#)
 2. [Trading concepts from zero](#)
 3. [Design philosophy](#)
 4. [The DB schema — your viewer contract](#)
 5. [Python for a C programmer](#)
 6. [Class-by-class walkthrough](#)
 7. [The decision pipeline \(one bar, step by step\)](#)
 8. [The ML part explained honestly](#)
 9. [Tuning knobs & how to modify things by hand](#)
 10. [Operations: running, restarting, watching](#)
 11. [Roadmap for your own improvements](#)
-

1. What this bot actually does

Every hour (one "bar"), the bot:

1. Downloads the latest price candle from Coinbase and stores it in SQLite.
2. Computes ~14 statistical features from recent price history.
3. Asks two ML models "will price go up, down, or nowhere?" and a third model "should we trust that answer?"
4. Checks risk limits (daily loss, max drawdown, market regime).
5. If everything aligns: opens a position with a pre-computed stop-loss and take-profit. Otherwise: does nothing, and logs *why* it did nothing.
6. Writes every single thought and action to SQLite so you can replay it.

In **paper mode** (default), step 5 is simulated with realistic fees and slippage — no real money moves. That's your 2-month dry run.

2. Trading concepts from zero

Candle (OHLCV). One time slice of the market. For a 1-hour candle: `(open)` = price at the start of the hour, `(high)`/`(low)` = extremes during the hour, `(close)` = price at the end, `(volume)` = how much was traded. Think of it as a struct sampled at a fixed rate:

```
c  
  
struct candle { time_t ts; double o, h, l, c, v; };
```

Position / Long / Short. A *position* means you hold an open bet. *Long* = you bought, you profit if price rises. *Short* = you sold something you don't own (borrowed), you profit if price falls. On a spot exchange like Coinbase you can't really short — the bot only shorts in paper mode, for research.

Stop-loss / Take-profit. Exit prices decided *at entry time*. Stop-loss: "if price falls to X, sell immediately, accept the small loss." Take-profit: "if price rises to Y, sell, lock in the win." This converts an open-ended gamble into a bounded one. The bot places both on every trade — no exceptions. Positions without stops are how accounts die.

Fees & slippage. Coinbase takes ~0.6% of every trade (taker fee). *Slippage* = the difference between the price you saw and the price you actually got (markets move while your order travels). Together these are the house edge against you. **This is the single most important number in the whole system:** if your average win per trade is smaller than fees + slippage, you lose money even with a good model. It's why the bot trades 1-hour bars, not 1-minute bars — bigger moves per trade, fewer fees paid.

ATR (Average True Range). The average size of recent candles — a volatility yardstick. "Stop at 2×ATR" means "give the trade room to breathe proportional to how wild the market currently is." A fixed stop like "-1%" would be too tight in wild markets and too loose in calm ones.

Drawdown. How far your account has fallen from its all-time peak, in percent. If you had 10'000 and now have 9'200, drawdown = 8%. The kill switch triggers at 10%.

Sharpe ratio. Return divided by volatility of returns — "profit per unit of stress." Above ~1.0 annualized after fees is genuinely good. Random strategies hover around 0. **This is the number your 2-month dry run must prove.** Sortino is the same but only counts downside volatility (nobody complains about upside swings).

Regime. Markets alternate between *trending* (prices move directionally — models can work) and *chopping* (random noise around a level — every model loses money to fees). The regime filter measures trend strength (ADX indicator) and refuses to trade in chop. This one filter probably matters more than the entire ML stack.

Why prediction is nearly impossible — and where the edge hides. Prices are ~95% noise. If anyone finds a pattern, trading it erases it (efficient market). The realistic goal is not "predict the price" but: find a *tiny* statistical edge (say 53% win rate at 1.5:1 reward/risk), apply it only in favorable regimes, size positions so no single trade matters, and let it compound. Most of this

bot's machinery — regime filter, meta-model gate, risk limits — exists to *avoid trading*, not to trade. The dry run tells you if the residual edge survives fees.

3. Design philosophy

Crash-only design. You know this from qmail/djb: there is no clean shutdown path that matters. The process may be SIGKILLED at any moment and must recover purely from persistent state. So: every state mutation goes to SQLite *immediately* (position opened → row committed *before* anything else happens). On start, the constructor re-reads the open position, equity, counters and kill-switch flag from the DB. There is no in-memory state that isn't reconstructible.

SQLite as the single source of truth *and* the IPC mechanism. WAL mode means one writer + many readers with no locking drama. Your qjs-glfw viewer is just another reader polling the file. No sockets, no protocol, no serialization format to design — the schema *is* the protocol. This is the same instinct as using the filesystem as an API.

One process, one loop, no threads. Everything happens sequentially inside `on_bar()`. On a 1-hour timeframe there is zero need for concurrency; the bot is idle 99.9% of the time. Threads would only add failure modes.

Adapter pattern for exchanges. `ExchangeAdapter` is an abstract interface (think vtable / function-pointer struct). `PaperAdapter` and `CoinbaseAdapter` implement it. The trading logic never knows which one it's talking to — so the paper run exercises the *identical* code path as live trading. Later, a `ForexAdapter` (OANDA) slots in the same way.

The bot logs its reasoning, not just its actions. Every HOLD comes with a reason string ("gate: dir=0.412 meta=0.551 regime=False"). When you replay a bad week in the viewer, you can see not just *what* it did but *why* — which threshold blocked which trade.

4. DB schema

All timestamps are **unix seconds, UTC** — directly usable as an x-axis. All tables are flat, no joins needed for rendering. WAL mode: open the DB read-only from QuickJS and poll; the bot's writes never block you.

`candles` — the market

column	type	meaning
ts	INTEGER	bar open time (unix sec)
product	TEXT	e.g. "BTC-EUR"
open,high,low,close	REAL	prices

column	type	meaning
volume	REAL	traded volume

PK `((product, ts))` — re-inserts overwrite, so backfill is idempotent.

`predictions` — what the bot *thinks*, every bar

column	meaning
p_long / p_flat / p_short	model probabilities, sum ≈ 1.0
meta_p	meta-model's confidence that the signal is trustworthy (0..1)
regime_ok	1 if the regime filter allowed trading this bar
model_ver	which training checkpoint produced this (e.g. "20260702-0300")

Viewer idea: draw p_long as a green line, p_short red, below the chart; shade the background grey where regime_ok=0.

`actions` — what the bot *did* (or deliberately didn't)

One row per decision. `(action) ∈ (BUY SELL HOLD HALT STOP TP SKIP)`. `(reason)` is free text meant for humans. `(confidence)` = meta_p at that moment. This is the table to render as markers/annotations on the chart.

`trades` — the position ledger

Row inserted with `(status='OPEN')` at entry, updated in place at exit (`(ts_close, exit, pnl, fees, status='CLOSED')`). At most one OPEN row per product — that invariant is what makes restart-reconcile trivial. `(stop)` and `(tp)` are stored so the viewer can draw the bracket lines.

`equity` — the scoreboard

One row per bar: mark-to-market `(equity)` (includes unrealized P&L of an open position), `(drawdown)` from peak, `(day_pnl)`. This is your main "is it working" curve.

`events` — the syslog

`((ts, level, msg))` with level INFO/WARN/ERROR. Kill-switch trips, training runs, restarts, exceptions all land here.

`state` — key-value store, JSON values

`(equity, equity_peak, day, day_start_eq, halted, bars_since_train)`. This is the crash-recovery state. `(halted=true)` survives restarts — that's deliberate: a kill-switched bot must not

resurrect itself.

Replay algorithm for your viewer: `(SELECT MIN(ts), MAX(ts) FROM candles)` → step a cursor bar by bar → at each ts, fetch the candle, the prediction, any actions, the equity point. Live mode: same, but poll `(MAX(ts))` every few seconds and append.

5. Python for a C programmer

The mental remapping you need, nothing more:

No declarations, dynamic typing. `(x = 5)` — the *name* is untyped, the object has a type. Think `(void*)` everywhere with runtime type tags, plus refcounting GC (CPython literally is refcounted, like your qjs objects).

Indentation = braces. The single biggest hazard when editing by hand. A block is defined by its indent level (the codebase uses 4 spaces). Dedenting a line *moves it out of the block*. There is no `}` to save you. When you edit, keep your editor showing whitespace.

`(self)` = explicit `(this)`. A method `(def foo(self, x))` is exactly `(int Class_foo(Class *self, int x))`. Calling `(obj.foo(5))` passes `(obj)` as `(self)` automatically. Python classes are essentially structs with a function table — no hidden magic.

`(__init__)` = constructor. Runs at `(Bot(cfg))`. `(__name__ == "__main__")` at the bottom is the Python idiom for "am I the executable, not a library" — same purpose as your `(main())` convention in qjs.

`@dataclass` (on `(Config)`) auto-generates the constructor from the field list — it's a struct definition with defaults, nothing more.

Exceptions instead of return codes. `(try:/except Exception as e:)` is the error path. The main loop wraps `(on_bar())` in one so a single bad bar can't kill the process — errors go to the `(events)` table and the loop continues. Uncaught exceptions terminate (then systemd restarts, then the DB restores state — crash-only, so this is fine).

numpy / pandas — the part that will feel alien. These are vectorized array libraries; the loop is *inside* the C implementation, not in your code:

```
python
f["ret1"] = logc.diff(1)           # whole-column op, no visible loop
```

is conceptually

```
c
for (i = 1; i < n; i++) out[i] = logc[i] - logc[i-1];
```

A pandas `DataFrame` = a table of named columns (each a typed array). `df["close"]` picks a column, `.iloc[-1]` = last element (negative indices count from the end, like `arr[n-1]`), `.rolling(20).mean()` = sliding-window average, `.ewm(...)` = exponential moving average (IIR one-pole filter — you know these from DSP: $y[i] = a \cdot x[i] + (1-a) \cdot y[i-1]$). `.dropna()` removes rows where a window wasn't full yet (the first 19 rows of a 20-window have no value — pandas marks them NaN instead of garbage).

PyTorch tensors = numpy arrays + autodiff + GPU. `with torch.no_grad():` = "inference only, don't record the computation graph" (cheaper). `.item()` unboxes a 1-element tensor to a float.

String formatting: `f"pnl={pnl:.2f}"` = `printf("pnl=%.2f", pnl)`.

Dicts `{"key": val}` = hash maps, used here like ad-hoc structs (e.g. the `trade` dict).

`t["entry"]` = field access.

That's 95% of what you'll meet in this file.

6. Class-by-class walkthrough

Config — every tunable in one struct. Change values here, nowhere else. API keys come from environment variables (`CB_API_KEY`, `CB_API_SECRET`) so they never live in the file.

Database — thin sqlite3 wrapper. `SCHEMA` is executed with `IF NOT EXISTS` on every start (idempotent migration-lite). Writers commit immediately (durability over throughput — at 1 write/hour who cares). `get`/`set` implement the JSON key-value `state` table.

ExchangeAdapter / **CoinbaseAdapter** / **PaperAdapter** — the vtable.

`CoinbaseAdapter.fetch_candles` chunks requests (API cap: 350 candles per call) and sleeps 250ms between chunks for rate limits. `PaperAdapter` *delegates* candle fetching to the real adapter (real market data!) but fakes fills: fill price = close ± slippage, fee = 0.6%. The pessimistic fee assumption is intentional — if the strategy survives 0.6%, reality will only be kinder.

FeatureEngine — turns raw candles into the 14-column feature matrix (section 8 explains what each feature means). Also provides `atr_abs()` (for stop distances) and `adx()` (for the regime filter). All pure functions of the candle DataFrame — no state.

TripleBarrierLabeler — only used during training. For every historical bar it asks: within the next 12 bars, what happened *first* — price rose $1.5 \times \text{ATR}$ (label 2), fell $1.5 \times \text{ATR}$ (label 0), or neither (label 1)? This produces labels that correspond to *tradeable outcomes* (would the TP or the SL have been hit?) instead of the naive "was the next candle green?" which is pure noise.

AttnLSTM — the neural net. See section 8.

ModelEnsemble — owns all three models, training and inference. Checkpoints are swapped atomically (`write tmp → os.replace`) — the same rename-is-atomic trick you'd use in C. Feature normalization (mean/std) is computed on the training split only and saved alongside the

models — inference must use the *same* normalization as training or the numbers are meaningless.

RegimeFilter — two-line gate: $ADX > 20$ (trend exists) AND $ATR > 2.5 \times$ the fee rate (moves are big enough that fees don't eat the edge). Returns bool.

RiskManager — all money-safety logic. Tracks equity/peak/day counters, persists them on every change. `position_size()` implements volatility targeting: risk a fixed 1% of equity per trade, so position size shrinks automatically when the market gets wild. Hard cap: never more than 25% of equity in one position. The kill switch (`halted`) flips at 10% drawdown and stays flipped across restarts.

Reporter — computes Sharpe, Sortino, max drawdown, win rate, profit factor (gross wins / gross losses), total fees. Run `python tradebot.py report` any time.

Bot — the orchestrator. Constructor = restart logic (reload open position, counters, install SIGTERM/SIGINT handlers). `run()` = the loop: do a bar, write heartbeat file, sleep until 5 seconds after the next candle close. `on_bar()` = the pipeline below.

7. Decision pipeline

What happens on every bar, in order — this is `Bot.on_bar()`:

backfill()	— fetch any missing candles (heals downtime gaps)
check stops	— did the last candle's high/low cross the open position's stop or TP? → close at that price
roll day	— new UTC day? reset day counters, log daily report
retrain?	— every 24 bars: retrain all models on full history, atomic checkpoint swap
predict	— LSTM + XGBoost → p_long/p_flat/p_short (blended) meta-XGB → meta_p ("trust this signal?") → row into `predictions` (always, even on HOLD)
risk gate	— halted? daily loss limit hit? → HALT (close pos.)
entry gate	— no position open AND regime_ok AND max(p_long, p_short) ≥ 0.45 AND meta_p ≥ 0.58 → enter, with stop = entry-2·ATR, tp = entry+3·ATR else → HOLD with reason string
exit-on-flip	— position open, opposite prob > 0.55 & meta ok → close early
mark-to-market	— equity incl. unrealized P&L → `equity` table

Note the asymmetry: entering requires *four* conditions to align; exiting requires only one (stop, TP, flip, or halt). Easy out, hard in.

8. The ML part

The 14 features, in plain words. `(ret1/3/6/12)` — log returns over 1/3/6/12 bars (momentum at several horizons). `(rsi)` — is the market overbought/oversold (bounded oscillator). `(macd_h)` — momentum of momentum (difference of two EMAs, then its own trend). `(bb_pos)` — where is price inside its recent 2-sigma band. `(atr_rel)` — current volatility relative to price. `(vol_z)` — is trading volume unusual right now (z-score vs. 48-bar history). `(vwap_d)` — distance from the volume-weighted average price (institutional reference level). `(hod_sin/cos, dow_sin/cos)` — hour-of-day and day-of-week encoded on a circle, so the model can learn "Sunday nights are dead" without a discontinuity at midnight (23→0 is a small step on the circle, not a jump of 23).

Why three models?

XGBoost (primary direction) — gradient-boosted decision trees. On tabular, noisy, small-ish financial data this is empirically the strongest tool, better than deep nets most of the time. It sees only the current bar's 14 features.

AttnLSTM — sees the *sequence* of the last 64 bars, so it can catch temporal shapes XGBoost can't. Attention pooling means: instead of using only the final LSTM state, learn a weighting over all 64 timesteps and take the weighted sum — the net decides which past bars matter. Trained with early stopping (stop when validation loss stops improving — the standard overfitting brake), dropout, weight decay, gradient clipping. Output: 3-class softmax.

The two direction opinions are blended 50/50.

Meta-XGB (the gate) — trained on a different question: "given the features AND the primary model's own output, was the primary model *right*?" This is López de Prado's *meta-labeling*: separate the "which direction" question from the "should I bet at all" question. It's the single most effective trick in the file — most of the primary model's signals are garbage, and the meta model's job is to identify the minority that aren't.

Training hygiene. The data is split 85/15 *chronologically* (never randomly — random splits leak the future into the past and produce fantasy accuracy). Normalization statistics come from the train split only. Labels use the triple-barrier method so they encode tradeable outcomes. Retraining runs every 24 bars on full history and swaps checkpoints atomically — a crash mid-training leaves the old model intact.

What to expect, honestly. Validation accuracy will look unimpressive (~40–50% on 3 classes; 33% is chance). That's normal and fine — the money is made by the *gate* being selective and the reward/risk being 3:2 (TP at $3\times\text{ATR}$, stop at $2\times\text{ATR}$), not by raw accuracy. Distrust any change that makes validation accuracy jump dramatically; you probably leaked future data.

9. Tuning knobs

All in `Config`. The ones you'll actually touch:

knob	default	effect of changing
<code>granularity</code>	ONE_HOUR	SIX_HOUR = fewer, bigger, cheaper trades; FIFTEEN_MINUTE = more trades, fees hurt more. Also update <code>tf_sec</code> mapping in <code>main()</code> — it's automatic there.
<code>entry_meta_min</code>	0.58	The main lever. Higher = fewer, better trades. Try 0.55–0.65 in replay.
<code>entry_dir_min</code>	0.45	Direction confidence floor.
<code>stop_atr / tp_atr</code>	2.0 / 3.0	The reward:risk geometry. Widening the stop reduces stop- outs but each loss costs more.
<code>target_vol</code>	0.01	Risk per trade as fraction of equity. Leave at 1% until the dry run proves an edge.
<code>max_drawdown</code>	0.10	Kill-switch threshold.
<code>retrain_every_bars</code>	24	Retraining cadence.
<code>barrier_atr / horizon</code>	1.5 / 12	Label geometry for training. Should roughly rhyme with stop/tp geometry.

Recipes for hand modifications:

Add a feature — three edits in `FeatureEngine`: append a name to `COLS`, compute the column in `compute()` (follow any existing line as a template — one pandas expression), done. Both models pick it up at the next retrain automatically because everything is dimensioned off `len(FeatureEngine.COLS)`. Delete the `models/` directory to force clean retraining after changing features (old checkpoints have the old input width and will crash otherwise).

Add a second trading pair — currently `Config.product` is a single string. Cleanest path: run two bot processes with two configs and two DB files. Multi-product in one process means making `position`, predictions and the retrain counter per-product — doable, but do it after the dry run.

Change the entry logic — it's all in the middle of `Bot.on_bar()`, the block starting `elif self.position is None:`. It's ~6 lines of plain comparisons. E.g. to require volume confirmation, compute `vol_z` there and add it to the condition.

Add an exit rule (e.g. time-based: close after N bars) — in `on_bar()`, in the `else:` branch where a position is open:

```
python
```

```
if ts - self.position["ts_open"] > 24 * self.cfg.tf_sec:
    self._exit(ts, price, "TIMEOUT")
```

Trailing stop — in `_check_stops()`, before the stop test, ratchet the stop upward for longs:

```
python

new_stop = candle.close - self.cfg.stop_atr * atr_abs    # pass atr in
if p["side"] == "LONG" and new_stop > p["stop"]:
    p["stop"] = new_stop
    self.db.con.execute("UPDATE trades SET stop=? WHERE id=?",
                        (new_stop, p["id"])); self.db.con.commit()
```

(Persist the change — the viewer will then draw the ratcheting stop line, which looks great in replay.)

Telegram alerts — add ~10 lines in `Database.log_event`: if level is ERROR,

```
urllib.request.urlopen("https://api.telegram.org/bot<TOKEN>/" "sendMessage?
chat_id=<ID>&text=" + urllib.parse.quote(msg))) wrapped in try/except. No library needed.
```

Python gotchas when editing by hand: indentation is structure (see §5); `=` vs `==` is caught at parse time, but `is` vs `==` is not (`==` for values, always); integer division is `//`, `/` always yields float; and mutating a dict you got from the DB does *not* write back — you must issue an UPDATE (see trailing-stop recipe).

10. Operations

```
sh

export CB_API_KEY=...  CB_API_SECRET=...
python tradebot.py backfill      # first run: pull 365 days of candles
python tradebot.py train        # initial model training (minutes)
python tradebot.py run           # the loop; Ctrl-C = clean stop
python tradebot.py report        # metrics as JSON, any time
python tradebot.py reset-halt    # ONLY after understanding why it halted
```

systemd unit (Restart=on-failure) as given earlier. The bot touches `tradebot.heartbeat` (a file containing the unix time) once per loop — a trivial cron watchdog can alert if it goes stale:

```
sh

# alert if heartbeat older than 2 bar-lengths
[ $(( $(date +%s) - $(cat tradebot.heartbeat) )) -gt 7200 ] && notify
```

Watching live from QuickJS (viewer preview, your house style):

```
js

import * as fs from 'fs';
// open read-only; WAL lets us read while the bot writes
// (via the sqlite binding from qjs-modules)
function lastAction(db) {
  const row = db.prepare('SELECT ts,action,reason,price FROM actions ORDER BY ts DESC LIMIT 1').get();
  console.log(new Date(row.ts * 1000).toISOString(), row.action, row.reason, row.price);
}
```

11. Roadmap

In order of expected impact per hour of your time:

1. **Run the dry run and stare at replays.** Nothing teaches faster than watching the bot lose money in slow motion in your own viewer.
2. **A proper backtest command** (`tradebot.py backtest`) that runs `on_bar()` over historical candles with the PaperAdapter — lets you test threshold changes in minutes instead of weeks. The pieces all exist; it's mostly a loop that feeds candles from the DB instead of the API.
3. **Walk-forward evaluation** — retrain on months 1-6, test on month 7, slide, repeat. The honest way to know if the edge is real.
4. **Threshold sweep on replay data** — grid over `entry_meta_min` × `stop_atr` × `tp_atr`, pick by Sharpe, but distrust the best cell (overfitting); prefer a plateau of good cells.
5. **More features** — funding rates, BTC dominance, order-book imbalance (needs the WebSocket feed). Features beat models.
6. **Forex adapter** (OANDA v20 practice account) — same interface, new `fetch_candles`/order methods, separate DB and models. Sessions and spread-based costs replace fees in the paper fill model.
7. **Portfolio mode** — multiple products, correlation-aware sizing. Only worth it once a single product shows an edge.

Last word: the machinery is now more disciplined than most retail bots. But discipline only guarantees you lose slowly and knowably — the edge itself must be proven by the dry run, and it's allowed to not exist. That result would still be worth two months: it's the difference between knowing and hoping.